

RELATÓRIO FINAL DE TESTES

Sistema Archivum

Engenharia de Software

Equipe Alexandre Rodrigues, Caio Murilo, Caian Dieter,
Ester Yasmin, André Luiz e Artur Braz.

Arquitetura

Backend (Python)

Construímos a API REST utilizando o framework **Flask**. Optamos por Python devido ao ecossistema robusto para testes. O banco de dados **SQLite** garante persistência relacional sem gargalos de infraestrutura.

Frontend (Vanilla)

A interface foi desenvolvida puramente em **HTML, CSS e JavaScript Vanilla**. Utilizamos a Fetch API nativa do navegador para nos comunicarmos de forma assíncrona com o servidor, garantindo renderização dinâmica.

Estratégia e Escopo de Testes



Testes Unitários

Validação da matemática pura do sistema. Verificamos as regras de negócio isoladas (como bloqueio de campos vazios) sem depender de banco ou servidor.



Testes de API

Isolamento do banco de dados em memória. Utilizamos o **Pytest** para disparar os métodos POST, GET, PUT e DELETE e validar os Status Codes.



Testes E2E

Automação completa do navegador utilizando **Playwright**. Simulação do usuário clicando na interface e verificando se o HTML foi alterado.

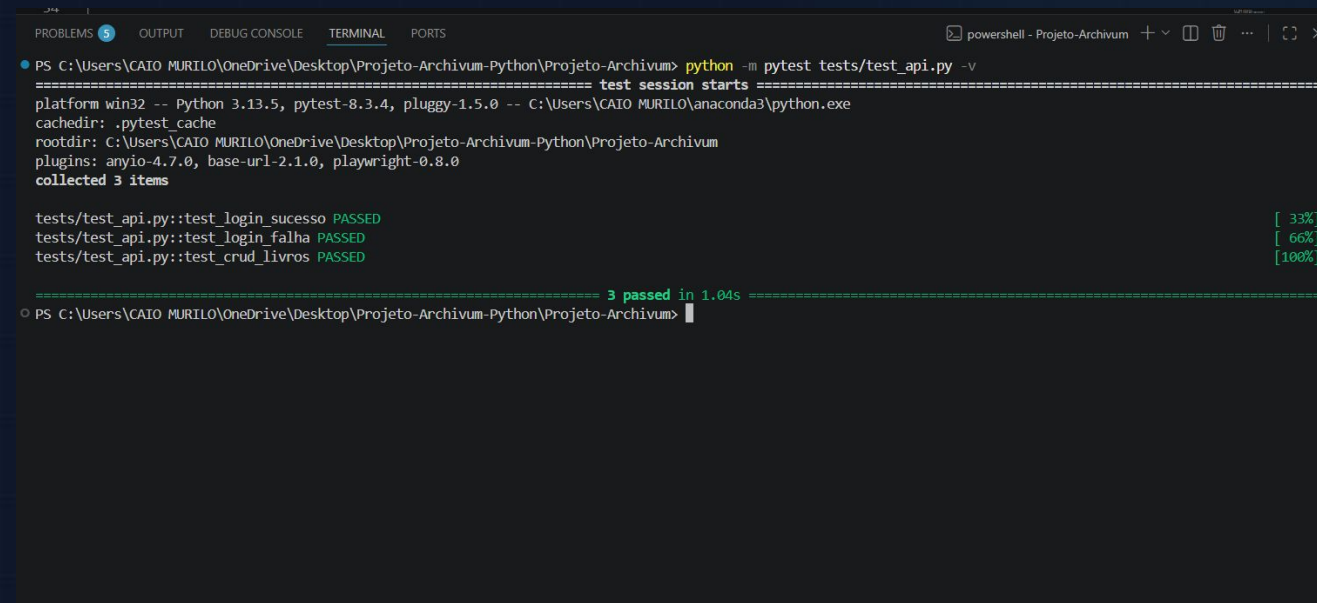
Mapeamento em BDD

- ✓ **História 1 - Autenticação:** *Dado que insiro credenciais válidas, Quando clico em Entrar, Então o sistema valida e redireciona para a aba do acervo.*
- ✓ **História 2 - Cadastro (Caminho Feliz):** *Dado que preencho título e autor, Quando salvo, Então a API persiste no banco e renderiza a lista.*
- ⚠ **História 3 - Regra de Negócio (Negative Testing):** *Dado que o usuário tenta salvar um livro com título vazio, Então a camada Unitária intercepta o erro e bloqueia a ação, retornando erro na requisição.*
- ✓ **História 4 - Deleção e Limpeza:** *Dado que o administrador exclui um livro, Então a requisição de DELETE remove o ID correspondente da base, sem afetar as demais.*

Evidências dos testes e execuções

Validação de login- Ct-01

- **Foco do Teste:** Validação de segurança e autenticação na camada de API (Backend).
- **Ação Realizada:** O script automatizado envia uma requisição POST para a rota /api/login injetando as credenciais do sistema.
- **Critério de Aceitação:** O sistema deve consultar o banco de dados, reconhecer o administrador e rejeitar usuários inválidos.
- **Resultado Obtido:** O servidor processou os dados com sucesso, retornando Status Code 200 (OK) e comprovando a estabilidade da rota.



```
PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum> python -m pytest tests/test_api.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-8.3.4, pluggy-1.5.0 -- C:\Users\CAIO MURILO\anaconda3\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum
plugins: anyio-4.7.0, base-url-2.1.0, playwright-0.8.0
collected 3 items

tests/test_api.py::test_login_sucesso PASSED [ 33%]
tests/test_api.py::test_login_falha PASSED [ 66%]
tests/test_api.py::test_crud_livros PASSED [100%]

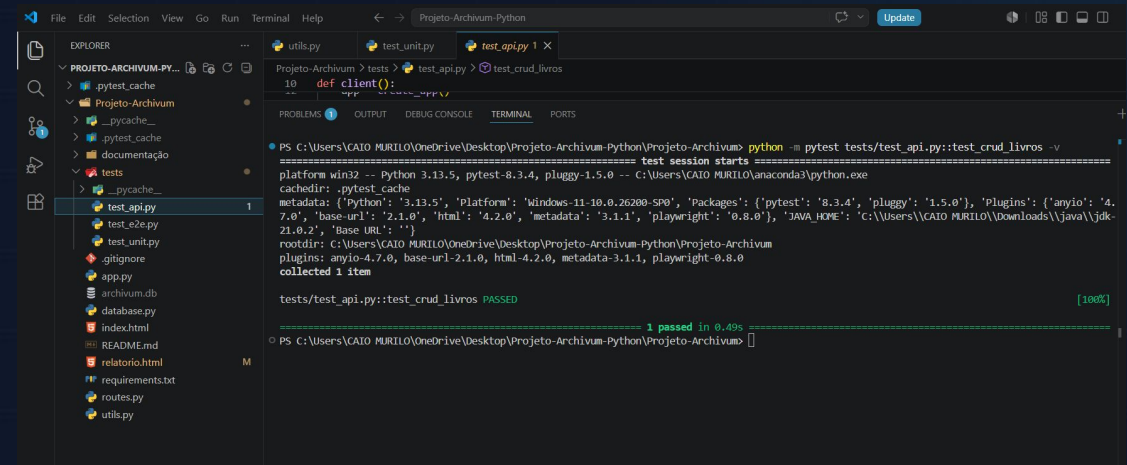
===== 3 passed in 1.04s =====
PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum>
```

Validação do Ciclo CRUD - CT-02

Testes de Rota no Pytest

O script test_api.py comprova tecnicamente o atendimento aos critérios de aceitação encadeando as 4 operações de banco:

- **POST (Create):** Status 201 gerado com sucesso.
- **GET (Read):** Retorno da lista de livros em JSON.
- **PUT (Update):** Identificador PK (ID) mantido na edição.
- **DELETE:** Limpeza do registro (Status 200).



```
31
32 def test_crud_livros(client):
33     """Testa o fluxo completo de Criar, Ler, Atualizar e Deletar um livro"""
34
35     # 1. CREATE (Registrar)
36     res_post = client.post('/api/livros', json={"titulo": "O Príncipe", "autor": "Maquiavel"})
37     assert res_post.status_code == 201
38     livro_id = res_post.json['id']
39
40     # 2. READ (Consultar)
41     res_get = client.get('/api/livros')
42     assert res_get.status_code == 200
43     assert len(res_get.json) > 0
44
45     # 3. UPDATE (Atualizar)
46     res_put = client.put(f'/api/livros/{livro_id}', json={"titulo": "O Príncipe Editado", "autor": "Maquiavel"})
47     assert res_put.status_code == 200
48
49     # 4. DELETE (Deletar)
50     res_del = client.delete(f'/api/livros/{livro_id}')
51     assert res_del.status_code == 200
```

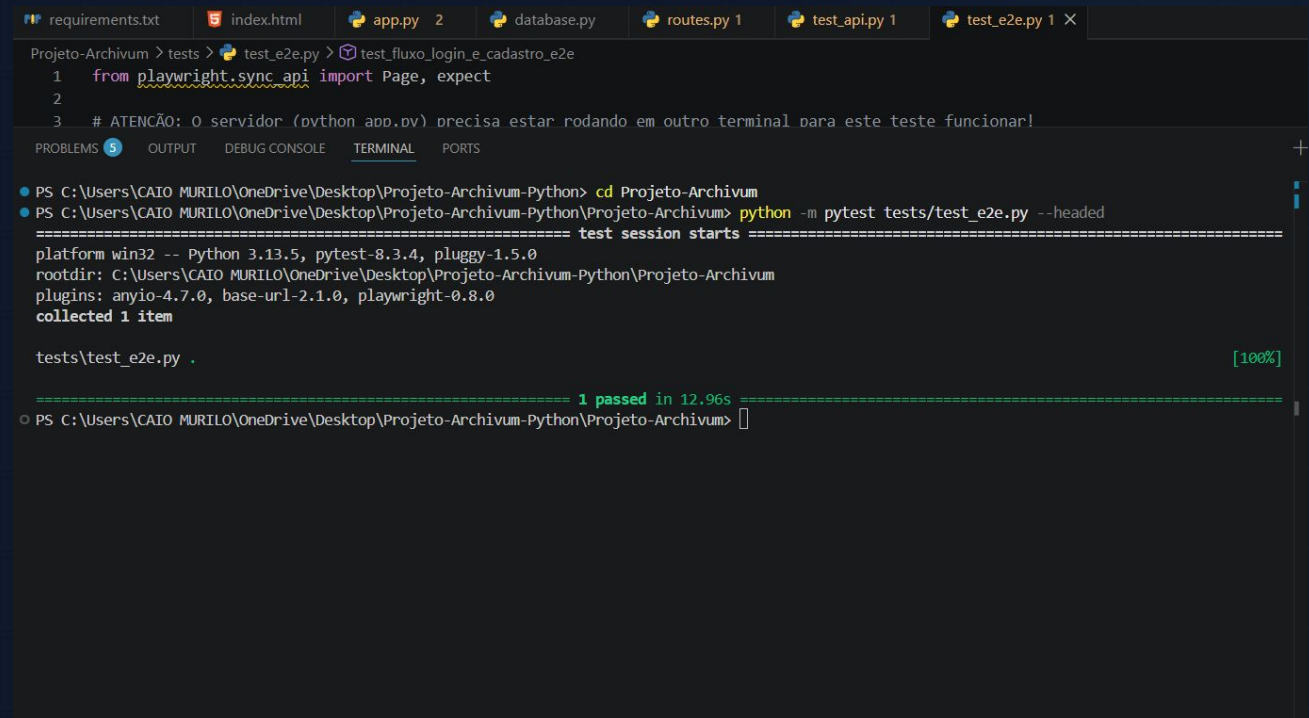
Automação de Interface (E2E)- Ct-03

Integração com Playwright

O teste End-to-End validou na prática se a integração Front/Back estava coesa.

O robô do Playwright interagiu com o HTML, acessou a URL localhost:3000, inseriu os dados nos inputs, realizou o login e cadastrou a obra literária.

Não houve exceções no console do navegador e o DOM renderizou o livro cadastrado perfeitamente.



The screenshot shows a VS Code editor with several files open: requirements.txt, index.html, app.py (2 tabs), database.py, routes.py (1 tab), test_api.py (1 tab), and test_e2e.py (1 tab). The active file is test_e2e.py, which contains the following code:

```
Projeto-Archivum > tests > test_e2e.py > test_fluxo_login_e_cadastro_e2e
1 from playwright.sync_api import Page, expect
2
3 # ATENÇÃO: O servidor (ovthon app.py) precisa estar rodando em outro terminal para este teste funcionar!
```

The terminal output shows the execution of the test:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python> cd Projeto-Archivum
• PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum> python -m pytest tests/test_e2e.py --headed
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-8.3.4, pluggy-1.5.0
rootdir: C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum
plugins: anyio-4.7.0, base-url-2.1.0, playwright-0.8.0
collected 1 item

tests\test_e2e.py . [100%]

===== 1 passed in 12.96s =====
o PS C:\Users\CAIO MURILO\OneDrive\Desktop\Projeto-Archivum-Python\Projeto-Archivum>
```


Vídeos dos testes



Vídeo 1: O Sistema

Demonstração manual do Login, e uso da tela para as operações de CRUD no navegador.



Vídeo 2: Pytest (API)

Execução ultra-rápida no terminal isolando o banco de dados e validando o Unitário e as Rotas.

Vídeos dos testes



Vídeo 3: Playwright

O robô E2E abrindo a janela gráfica e interagindo com o DOM em alta velocidade.



Vídeo 4: Pytest para Teste unitário

Execução do pytest para testar funcionalidades de forma isolada/focada.

Conclusão e Métricas

100
PASSED
%

0 Falhas Registradas na Automação

O sistema **Archivum** foi aprovado com êxito. A separação clara entre a persistência de dados (SQLite), as regras de negócio e a interface viabilizou uma esteira de testes veloz e determinística.

O relatório HTML gerado comprova a robustez das 3 camadas exigidas, assegurando que futuras atualizações de código não gerarão problemas de regressão.

Obrigado(a)

pela atenção!

 <https://github.com/CaioMurilo/projeto-archivum-testes>